PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System

Yintao He
SKLP, Institute of Computing Technology, CAS
University of Chinese Academy of Sciences
Beijing, China

Haiyu Mao
King's College London
ETH Zürich
London, United Kingdom

Christina Giannoula
University of Toronto
Vector Institute
ETH Zürich
Toronto, Canada

Mohammad Sadrosadati
SAFARI Research Group
ETH Zürich
Zurich, Switzerland

Juan Gómez-Luna
NVIDIA
Zurich, Switzerland

Huawei Li
SKLP, Institute of Computing Technology, CAS
University of Chinese Academy of Sciences
Beijing, China

Xiaowei Li
SKLP, Institute of Computing Technology, CAS
University of Chinese Academy of Sciences
Beijing, China

Ying Wang
SKLP, Institute of Computing Technology, CAS
Beijing, China

Onur Mutlu
SAFARI Research Group
ETH Zürich
Zurich, Switzerland

Abstract

Large language models (LLMs) are widely used for natural language understanding and text generation. An LLM model relies on a time-consuming step called LLM decoding to generate output tokens. Several prior works focus on improving the performance of LLM decoding using parallelism techniques, such as batching and speculative decoding. State-of-the-art LLM decoding has both compute-bound and memory-bound kernels. Some prior works *statically identify* and map these different kernels to a heterogeneous architecture consisting of both processing-in-memory (PIM) units and computation-centric accelerators (e.g., GPUs). We observe that characteristics of LLM decoding kernels (e.g., whether or not a kernel is memory-bound) can change *dynamically* due to parameter changes to meet user and/or system demands, making (1) *static* kernel mapping to PIM units and computation-centric accelerators suboptimal, and (2) one-size-fits-all approach of designing PIM units inefficient due to a large degree of heterogeneity even in memory-bound kernels.

In this paper, we aim to accelerate LLM decoding while considering the dynamically changing characteristics of the kernels involved. We propose PAPI (**PA**rallel Decoding with **PI**M), a PIM-enabled heterogeneous architecture that exploits dynamic scheduling of compute-bound or memory-bound kernels to suitable hardware units. PAPI has two key mechanisms: (1) *online kernel characterization* to dynamically schedule kernels to the most suitable hardware units at runtime and (2) a PIM-enabled heterogeneous computing system that harmoniously orchestrates both computation-centric processing units (GPU) and hybrid PIM units with different computing capabilities. Our experimental results on three broadly-used LLMs (i.e., LLaMA-65B, GPT-3 66B, and GPT-3 175B) show that PAPI achieves 1.8× and 11.1× speedups over a state-of-the-art heterogeneous LLM accelerator (i.e., GPU and PIM) and a state-of-the-art PIM-only LLM accelerator, respectively.

CCS Concepts: • **Computer systems organization** → **Parallel architectures**; • **Computing methodologies** → *Machine learning*.

Keywords: Processing-in-Memory; Large Language Models; Heterogeneous Systems; Dynamic Scheduling; Generative Artificial Intelligence

ACM Reference Format:
Yintao He, Haiyu Mao, Christina Giannoula, Mohammad Sadrosadati, Juan Gómez-Luna, Huawei Li, Xiaowei Li, Ying Wang, and Onur Mutlu. 2025. PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25), March 30-April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3676641.3716009

This work is licensed under a Creative Commons Attribution 4.0 International License.
ASPLOS '25, Rotterdam, Netherlands
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1079-7/2025/03
https://doi.org/10.1145/3676641.3716009

1 Introduction

Large language models (LLMs) have achieved remarkable success across a wide range of applications, excelling not only in natural language processing tasks such as code generation [9, 60], question answering [13, 79], but also image [78, 80, 82] and video processing [70]. Efficient LLM inference is crucial to unlocking the full potential of these models. LLM inference consists of two phases: prefill and decoding [104]. In the prefill phase, the LLM model processes all input tokens in a request to create hidden states for the decoding phase and generate the first output token. Subsequently, during the conventional decoding phase, the model generates an output token per decoding iteration.

Low execution time of LLM inference is crucial for both user experience in real-time applications and hardware utilization efficiency [89]. The LLM decoding phase dominates the execution time in the LLM inference tasks [34, 103]. For instance, the serial decoding of the GPT-3 175B model is responsible for 96% of the overall execution time when the input and output lengths are 32 [11]. The impact of LLM decoding on overall execution time increases as the output length grows, which is essential for generating more detailed and comprehensive LLM responses [5].

To improve the performance of the decoding phase, prior works employ two main parallelism techniques: batching [3, 76, 101] and speculative decoding [7, 51, 90, 102]. These techniques enable the generation of multiple tokens, known as *parallel decoding*, in one decoding iteration, to accelerate decoding. We define the number of decoding tokens that are simultaneously generated as *decoding parallelism*. Decoding parallelism directly affects the utilization of memory and computation resources. As a result, some kernels in decoding become compute-bound, while others become memory-bound. Recent works explore processing-in-memory (PIM) enabled hybrid designs (i.e., heterogeneous architectures using both PIM units and computation-centric accelerators, like GPUs) [33, 52, 73, 74, 84] to accelerate the LLM inference process by mapping compute-bound and memory-bound kernels to computation-centric and memory-centric accelerators. These works *statically* characterize LLM decoding kernels and, based on statically-identified characteristics, schedule different types of kernels to different computation units (e.g., PIM units and computation-centric accelerators like GPUs).

To study the effectiveness of static scheduling, we profile the kernels used in the decoding phase that employs state-of-the-art parallelism techniques. We observe that some kernels shift from being compute-bound to memory-bound (or vice versa) *dynamically* in response to variations in decoding parallelism. This is because decoding parallelism changes dynamically at runtime. There are three main reasons for these dynamic changes in decoding parallelism. First, the maximum decoding parallelism in a computing system is limited by the memory requirement of requests, which is dependent on request output lengths and cannot be predicted prior to execution. Second, the maximum decoding parallelism is also affected by different user requirements, like quality of service (QoS) [2]. Third, some parallelism techniques [58, 76] employ dynamic optimization approaches that adjust the configuration of decoding parallelism (i.e., batch size and speculation length) at runtime to enhance system performance (see Section 3). We conclude that dynamic changes in decoding parallelism cause heterogeneous designs with a *static* scheduling scheme to become suboptimal, as they can mistakenly schedule computation-intensive kernels to PIM units or memory-intensive kernels to computation-centric accelerators (e.g., GPUs).

In this paper, we aim to accelerate parallel decoding by fully leveraging the dynamically changing parallelism properties in LLM inference tasks. To this end, we propose PAPI (**PA**rallel Decoding with **PI**M), a PIM-enabled heterogeneous architecture that exploits dynamic scheduling of compute-bound or memory-bound kernels to the most suitable hardware unit for each kernel type. PAPI's key idea is to enable *online* dynamic task scheduling on a heterogeneous architecture (consisting of GPUs and PIM units) via online identification of kernel properties in LLM decoding.

PAPI is equipped with three key techniques. First, we propose *a dynamic parallelism-aware task scheduling framework* to assign kernels to suitable computing platforms at runtime. This approach employs a simple yet effective *kernel bottleneck predictor* with low hardware overhead. Second, we design *a heterogeneous architecture with PIM units, GPU, and host CPU* to meet different computing and memory demands of different kernels. Third, we design a hybrid PIM architecture that includes two different types of PIM units, i.e., *performance-optimized and memory-capacity-optimized PIM units*, which cater to memory-intensive kernels with different computational demands and memory footprints.

We compare PAPI with the state-of-the-art heterogeneous LLM accelerator composed of AttAcc [74] and 6 NVIDIA A100 GPUs [12] (A100+AttAcc), a heterogeneous architecture composed of Samsung's HBM-PIM [48] and the NVIDIA A100 GPUs (A100+HBM-PIM) and a PIM-only LLM accelerator, AttAcc [74]. Our experimental results show that PAPI outperforms A100+AttAcc, A100+HBM-PIM, and AttAcc by 1.8×, 1.9×, and 11.1×, respectively.

This paper makes the following contributions.

- We observe that parallelism in LLM decoding changes *dynamically*, leading to varying demands in both computation capability and memory bandwidth.
- We propose PAPI, a PIM-enabled heterogeneous computing system, to meet the different computation and memory bandwidth demands by incorporating both memory-centric PIM units and computation-centric GPU and host CPU.

- We propose an online parallelism-aware scheduling technique that maps dynamically LLM decoding kernels with different and changing properties to the most appropriate hardware units, including hybrid PIM units.
- We evaluate PAPI and demonstrate that it provides significant performance and energy benefits over state-of-the-art computing systems for LLM inference.

2 Background

2.1 LLM Inference

An LLM structure contains several transformer-based decoders, as illustrated in Figure 1(a). Each decoder includes four kernels: QKV (Query, Key, and Value) generation, multi-head attention, projection, and feedforward networks [95]. These kernels can be divided into two types: fully-connected (FC) layers in orange and a multi-head attention layer in green. All kernels consist of general matrix-vector multiplication (GEMV) computations.

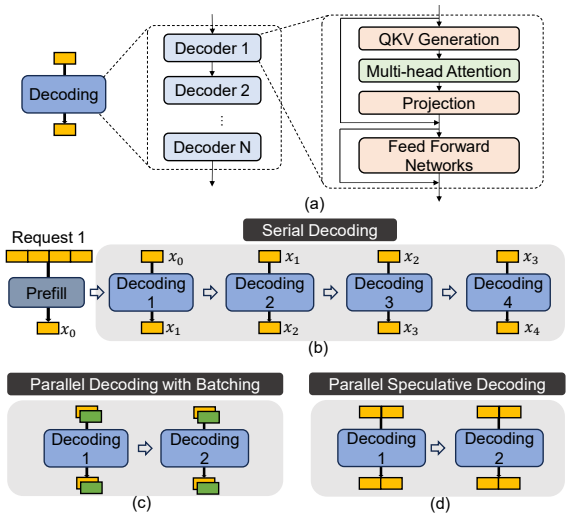

Figure 1. (a) LLM structure. (b) LLM inference with serial decoding. (c) Parallel decoding process with batching. (d) Parallel decoding process with speculative decoding.

Figure 1(b) presents an overview of LLM inference, which includes two phases: *prefill* and *decoding*. In the prefill phase, the model processes multiple tokens within the input sequence simultaneously to generate the first output token. The decoding phase has multiple decoding iterations. In each iteration, the model takes the last output token as input to generate a new output token. The output sequence is generated one by one sequentially until the <|eos|> (end of a sentence) token [95]. This serialized process is called *serial decoding*. Compared to the prefill phase, the decoding phase typically takes most of the end-to-end inference time [102]. The number of decoding iterations depends on the output token length. With serial decoding, generating K output tokens takes K decoding iterations [51].

In LLM inference, the output of QKV generation, i.e., K and V matrices, is stored as these output values are reused multiple times in the multi-head attention kernel during subsequent decoding iterations. In serial decoding, LLM inference requires GPUs/TPUs to load the large weight matrices and KV matrices from off-chip memory to on-chip memory/caches during *each* serial decoding iteration, causing high data movement and performance overheads.

2.2 Optimization Techniques in LLM Inference

To overcome performance overheads of conventional serial decoding, researchers have developed two optimization techniques: batching [3, 76, 95, 101] and speculative decoding [7, 51, 90, 102]. These methods facilitate the *concurrent* decoding of *multiple* tokens, thereby improving data reuse of weight matrices via generation of multiple tokens, which improves performance.

2.2.1 Batching. Batching [3, 76, 95, 101] is a parallelism technique to process multiple input sequences concurrently. It allows a single decoding step to generate multiple tokens from different user requests. This enables request-level parallelism (RLP) during inference, where RLP refers to the number of requests executed in parallel. For example, as shown in Figure 1(c), when an LLM processes two input requests simultaneously, RLP is 2. A state-of-the-art batching mechanism is *mixed continuous batching* [3, 76]. In mixed continuous batching, the system dynamically adjusts the number of requests in a batch at runtime based on available memory and computation resources, as well as the number of incoming requests. This technique allows new requests to be added to the current batch without waiting for all requests of the batch to be completed, thereby optimizing resource utilization and improving overall throughput.

2.2.2 Speculative Decoding. Speculative execution introduces a novel parallel decoding mechanism [7, 51], which includes two steps: *serial draft decoding* and *parallel speculative decoding*. First, a small draft model efficiently predicts the next 2-10 tokens, and these predicted tokens are then verified simultaneously with the large original LLM, allowing for the next tokens to be processed in parallel. Figure 1(d) shows this mechanism when two tokens are generated in one decoding iteration of the LLM. Speculative decoding enables token-level parallelism (TLP) in LLM inference. TLP represents the number of concurrently decoded tokens within a single decoding iteration. For example, two yellow tokens in Figure 1(d) for one request are decoded simultaneously, i.e., when TLP = 2.

3 Motivation

3.1 Analysis of LLM Inference

We analyze the computation and memory requirements of LLM inference by evaluating the arithmetic intensity of fully-connected (FC) and multi-head attention kernels. We vary the batch size, when batching is enabled, and speculation length, when speculative decoding is enabled. Figure 2(a) shows the roofline model for the OPT-30B model [6] using a high-end NVIDIA A100 GPU [12] with 312 TFLOPS peak computation performance and 1935 GB/s peak memory bandwidth, for FC and attention kernels, as the batch size increases from 4 to 128, with a speculation length of 8. We make two key observations. First, when the batch size is small, i.e., 4, 8, 16, the decoding phase is memory-bound, i.e., both the attention and FC kernels are bottlenecked by memory bandwidth. Second, when the batch size is ≥ 32, the FC kernel becomes compute-bound, while the attention kernel is memory-bound. The arithmetic intensity of the FC kernel increases with the batch size. However, the arithmetic intensity of the attention kernel does *not* change when the batch size increases, because there is no data reuse in batching for the attention kernel.

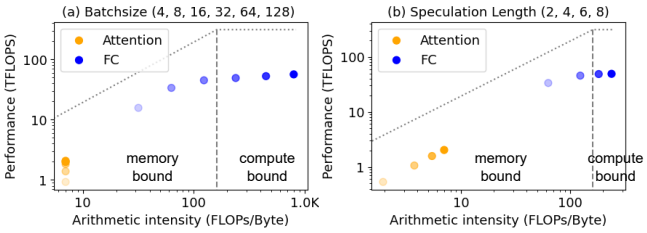

Figure 2. Roofline model using OPT-30B with (a) different batch sizes (speculation length = 8) and (b) different speculation lengths (batch size = 32). The darker the color of the dots, the higher the degree of parallelism.

Figure 2(b) shows the roofline analysis of FC and attention kernels when we vary the speculation length from 2 to 8, with a batch size of 32. We observe that the arithmetic intensity of the attention and FC kernels increases with the speculation length. With a batch size of 32, the FC kernel becomes compute-bound when the speculation length exceeds 6. In contrast, although the arithmetic intensity of the attention kernel increases with speculation length, the attention kernel remains memory-bound. This is because the arithmetic intensity of the attention kernel increases only slightly with speculation length, and batching has no effect on it, as batching primarily improves weight data reuse, which does not affect the attention kernel.

3.2 Varying Parallelization Levels in LLM Inference

In real-world LLM tasks, both batch size and speculation length vary significantly at runtime due to changes in user requests and potential adjustments to speculation length to optimize performance [58]. We elaborate on why the parallelism in LLM inference dynamically changes during runtime in real-world scenarios.

Initial Request-Level Parallelism (Initial RLP): Initial RLP refers to the RLP when batched execution begins. Initial RLP can vary significantly in real-world LLM serving scenarios, causing the batch size to vary greatly. This is due to three major reasons.

(a) *Service Level Objective (SLO) Limits*: Increasing RLP can enhance throughput but increases inference latency per request [55]. Under the online serving scenario, different user latency SLOs dictate varying maximum batch sizes. For example, while a DGX A100 computing system [68] with 1,280 GB memory can support up to 854 requests per batch, a 30 ms SLO requires setting the initial RLP to be as low as 22 [74].

(b) *Memory Capacity Limits*: Initial RLP is also constrained by the system's memory capacity, particularly for KV cache storage. A computing system with 640 GB HBM can house 282 requests with input and output lengths of 128, but only 18 requests with input and output lengths of 2048 [74]. In the latter case, the batch size needs to be smaller, as longer sequences need more memory capacity for KV cache for multi-head attention.

(c) *Dynamic Batching*: Dynamic batching [65] starts processing a batch once the batch is full or exceeds a time limit. Therefore, when requests are infrequent, an LLM serving system with dynamic batching may start processing with different batch sizes, and thus, different RLP values.

Runtime Request-Level Parallelism (Runtime RLP): Runtime RLP refers to the RLP during the execution of a batch of requests. Runtime RLP depends on the batching mechanism used, which may be *static* batching or *mixed continuous* batching [76].

Traditional LLM serving systems [65, 85] use static batching with batch-level scheduling. In this approach, no new requests are processed until all requests from the current batch have finished. Since each request has a unique output length, runtime RLP dynamically varies. As shown in Figure 3, runtime RLP dynamically decreases as each request of the current batch finishes (i.e., as more decoding iterations take place) [69].

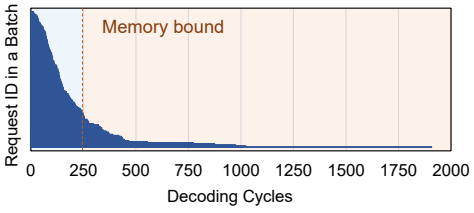

Figure 3. Decoding iterations required for each request in a batch, illustrating how the number of remaining parallel requests changes as decoding iterations increase.

Mixed continuous batching [3, 76] allows token-level scheduling, where new requests can be added to be processed by the LLM serving system, while the system is executing requests in the current batch. In this case, runtime RLP dynamically changes to keep the hardware resource utilization as high as possible, and it is dependent on when and how many requests are added to each batch.

Token-Level Parallelism (TLP): TLP can also be dynamically adjusted at runtime to enhance speculative decoding performance in LLMs [58, 91]. For instance, a prior work [58] introduces a dynamic speculation length optimization technique that adjusts speculation length during each decoding iteration. Additionally, batching and speculative decoding need to be synergistically co-optimized to improve GPU utilization for LLM inference [91]: e.g., when the batch size is small, the speculation length can be increased to maximize resource utilization.

We conclude that in real-world LLM serving scenarios, batch size and speculation length substantially vary during runtime. As a result, the arithmetic intensity of FC and attention kernels dynamically change due to the varying parallelization levels. Therefore, FC and attention kernels can become either compute-bound or memory-bound kernels, when executed in computation-centric systems such as GPUs. We draw two key insights from our analysis.

Key Insight 1. LLM serving requires a heterogeneous computing architecture with advanced computing units that offer varying computation throughput and memory bandwidth capabilities to satisfy the different arithmetic intensities of kernels.

Key Insight 2. LLM serving requires a *dynamic* scheduling approach to map FC kernels to different computing units because FC kernels can switch between being compute-bound or memory-bound during runtime.

3.3 Limitations of Existing Processing-In-Memory Architectures for LLM Inference

Computing units, such as GPUs and neural processing units (NPUs), are widely used for LLM serving systems. Recent works (e.g., [20, 33, 38, 44, 47, 52, 71, 73, 74, 84, 103]) explore the Processing-In-Memory (PIM) computing paradigm (e.g., [1, 4, 19, 30–32, 54, 86, 87]) in LLM inference to alleviate the data movement bottleneck in memory-bound kernels of LLMs, such as the attention kernel. By integrating processing cores within memory units, PIM provides high memory bandwidth, mitigating data movement bottlenecks in kernels with low arithmetic intensity.

Some prior works [33, 52, 73, 74, 84] propose PIM-enabled heterogeneous architectures for LLMs. These architectures include both high-performance computation-centric processors (e.g., GPUs) and PIM devices with very high memory access bandwidth. These works run LLM kernels in computation-centric processors or memory-centric PIM devices and demonstrate significant performance benefits compared to commodity systems, e.g., using only computation-centric accelerators (e.g., GPUs) to run end-to-end LLM inference. However, these prior works still suffer from two major shortcomings.

Shortcoming 1. *Prior works statically assign FC and attention kernels either to a computation-centric processor (GPU) or a PIM-enabled computing device.* Our analysis shows that *dynamic* assignment of kernels to different computing devices is necessary because LLM kernels have varying arithmetic intensity, e.g., FC kernel can be either compute-bound or memory-bound in GPUs, depending on the speculation length and batch size that are currently used. Specifically, AttAcc [74] always offloads all attention kernels to the proposed PIM devices and all FC kernels to a GPU. IANUS [84] statically maps all FC kernels to PIM and attention kernels to NPU. SpecPIM [52] proposes an allocation scheme that executes attention and FC kernels at the high-performance processor and PIM devices *concurrently*. However, it is only designed for a static batch size and speculation length. We conclude that these prior works do *not* sufficiently meet the varying computation and memory demands of real-world LLM serving scenarios. They propose *static* designs, where FC and attention kernels each are always mapped to the same computing hardware; even though kernels exhibit varying computation and memory demands at runtime.

To quantitatively demonstrate the limitations of prior PIM-based proposals for LLM inference, we evaluate the execution time (latency) of one FC kernel using an NVIDIA A100 GPU [12], Samsung's HBM-PIM architecture [48], and the state-of-the-art PIM-based work for LLMs, AttAcc [74] (Section 7 provides more detail on our evaluation methodology). Figure 4 shows the FC kernel latency (normalized to A100 GPU) when we vary the batch size and speculation length. We observe that in the configurations with low parallelization levels, e.g., having a batch size of 1 and speculation length of 8 or having a batch size of 4 and speculation length of 2, PIM-based architectures, i.e., HBM-PIM and AttAcc, provide better performance than the A100 GPU. In contrast, in the configurations with high parallelization levels, e.g., batch size of 16 or larger, the A100 GPU significantly outperforms

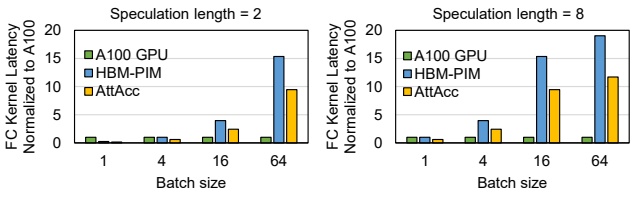

Figure 4. The normalized latency of the FC kernel in LLM inference with different parallelization levels (different batch sizes and speculation lengths).

the PIM-based architectures, providing much lower execution time. However, RLP and TLP are *not* known in advance (statically): they dynamically vary and it is hard to predict how they would change. This observation necessitates *dynamic* decisions of which computing hardware to use to execute the FC kernel.

Shortcoming 2. *Prior works support only one type of PIM-enabled computing device with a certain computation throughput and memory bandwidth capability.* Our analysis shows that although FC and attention kernels can be both memory-bound kernels in GPUs, necessitating PIM-enabled solutions, they have very different arithmetic intensities and different computation and memory bandwidth needs. For example, as demonstrated in Figure 2, with a batch size of 4 and speculation length of 8, the arithmetic intensity of FC is 31.7 FLOPs/Byte, while that of attention is 7.0 FLOPs/Byte. Thus, assuming computing hardware of a certain computation throughput, attention would need around 4.5× higher memory bandwidth than FC. This shows that PIM-enabled computing devices need to provide different computation and memory bandwidth capabilities to efficiently execute the two different types of LLM kernels. Our work is the first to identify this property of the two types of LLM kernels, while prior works use PIM-enabled devices with a fixed computation and memory bandwidth capability, which make them inefficient at meeting the different and dynamically varying needs of attention and FC kernels.

3.4 Our Goal

Our goal is to design a versatile computing platform that caters to the varying parallelization levels in real-world LLM inference with different and dynamically changing computation and memory demands. To this end, we propose (1) a heterogeneous architecture that integrates memory-centric PIM units and computation-centric GPU and host CPU, each offering distinct computation throughput and memory bandwidth characteristics, and (2) a parallelism-aware scheduling technique that adapts to runtime variations in parallelization and intelligently and dynamically assigns FC and attention kernels to the most appropriate hardware units in our platform.

4 PAPI: Overview

Given that LLM inference exhibits varying parallelization levels during runtime, an intelligent dynamic scheduling policy is necessary to identify the most suitable computing hardware for a given kernel at a given time. The **key challenge** is to design a kernel offloading and allocation scheme that monitors dynamic parallelism online at low cost (in terms of latency and energy consumption) and selects the best-fit computing hardware to fully and efficiently utilize the available hardware resources.

4.1 PAPI: Key Components

We propose the PAPI architecture and framework. Figure 5 shows the overview of the PAPI framework. PAPI has three key components explained next.

Heterogeneous Architecture. We propose a heterogeneous architecture to effectively cater to both compute-bound and memory-bound kernels of LLMs. This architecture includes (1) a host CPU, (2) a high-performance processor with PIM memory units (FC-PIM), and (3) physically separated (i.e., disaggregated) PIM units (Attn-PIM). The high-performance processor includes processing units (hereafter referred to as PUs), e.g., GPU tensor cores [67], PIM memory units (i.e., HBM-based PIM devices), and a hardware scheduler. In our evaluation, we use GPU tensor cores for the PUs, but any other high-performance processor designed for compute-bound kernels (e.g., TPU [39] or NPU [10]) could also be used for this design. The host CPU sends instructions to the high-performance processor and the physically separate Attn-PIM devices, which are disaggregated from the high-performance processor.

Hybrid PIM Units. We propose two types of PIM units to cater to the different parallelization levels of the FC and

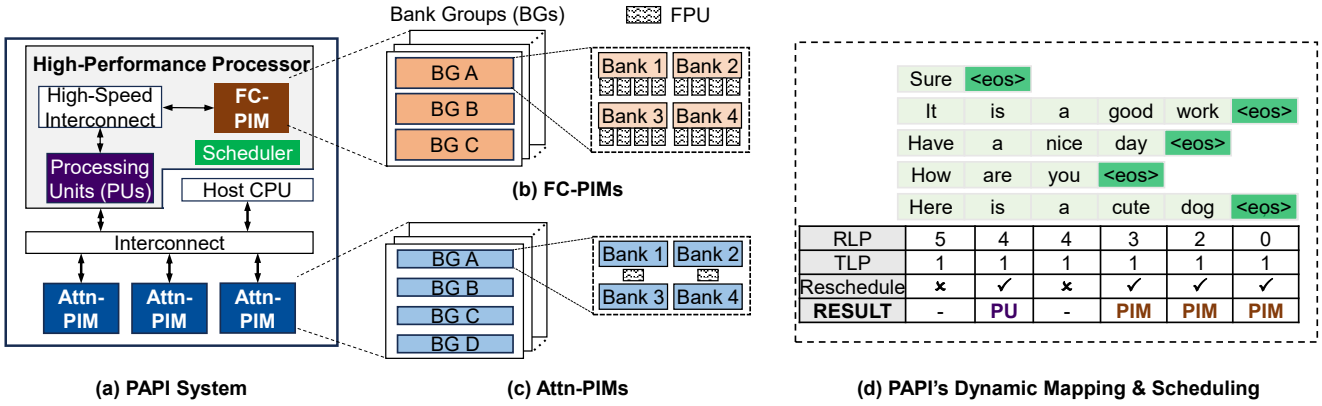

Figure 5. Overview of the PAPI computing system, and an example of its dynamic parallelism-aware scheduler.

attention kernels of LLMs. FC-PIM units offer relatively high computation capabilities to cater to the FC kernels, while Attn-PIM units provide a larger memory capacity tailored to the attention kernel. The hybrid PIM units are designed to overcome the limitations of prior existing PIM designs for LLMs (e.g., [45, 48, 74]), which typically support a single PIM unit type with fixed computation capabilities. PAPI separates FC and attention kernels across different PIM devices. Since attention kernels are always memory-bound, they are assigned to the Attn-PIM devices. FC kernels can be either compute- or memory-bound, and thus they can be dynamically allocated by the scheduler to either PUs or FC-PIM units.

Dynamic Parallelism-Aware Scheduling. As analyzed in Section 3.2, we need to identify whether or not the FC layer is memory-bound and dynamically offload it to the FC-PIM units or the PUs of the high-performance processor. Instead, the attention kernel is always memory-bound, only running on the Attn-PIM units. We introduce a hardware scheduler (green block in Figure 5(a)) that monitors runtime parallelization changes and implements dynamic scheduling. When the parallelization level changes, our scheduler executes a low-cost identification step, and offloads the FC kernel to the best-fit computing hardware. When the scheduler identifies the FC kernel as memory-bound, it executes FC on the FC-PIM devices. When it identifies FC as compute-bound, it executes FC on the high-performance processor PUs. In the latter case, FC-PIM memory units are used as main memory to keep the weight parameters, which are loaded and processed by the PUs. Figure 5(d) illustrates an example of PAPI's dynamic monitoring. Every time the parallelization level of the FC kernel changes, our dynamic monitoring framework is involved, identifying memory-bound or compute-bound kernels, and reallocating them to different units as needed.

5 PAPI Dynamic Scheduling

We propose an effective scheduling mechanism to offload FC kernels to PUs or FC-PIM units at runtime with low latency and low energy consumption. In this section, we first explain how the scheduling mechanism determines whether an FC kernel is memory-bound, and then provide the implementation details of the runtime scheduling.

5.1 Memory-Boundedness Identification of the FC Kernel

We identify whether or not the FC kernel is memory-bound by estimating its arithmetic intensity. Assume that the weight matrix dimensions of the FC kernel are (h, h) and the input given is $(RLP \times TLP, h)$, where h is the hidden dimension in the LLM structure. The arithmetic intensity of an FC kernel can be calculated as follows:

$$\text{AI} = \frac{\#\text{Flops}}{\#\text{Bytes}} = \frac{RLP \times TLP \times h^2 \times 2}{(2 \times RLP \times TLP \times h + h^2) \times 2} \quad (1)$$

In state-of-the-art LLMs, the hidden dimension h is typically large to support their advanced natural language processing tasks [13]. For example, $h = 12288$ in the GPT-3 175B model [6], and the arithmetic intensity can be estimated as follows:

$$\text{AI} \approx RLP \times TLP \quad (2)$$

Therefore, we can use $RLP \times TLP$ to estimate the arithmetic intensity of an FC kernel, where RLP and TLP are known at runtime.

To evaluate the accuracy of our arithmetic intensity estimation, we assess the FC kernel in the GPT-3 66B model using various RLP and TLP configurations. Figure 6 shows the actual obtained arithmetic intensity our estimated values. In most cases, our estimations very closely match the actual arithmetic intensity. When parallelization level is very large (e.g., RLP=128), the estimated value is slightly larger than the actual arithmetic intensity. In such cases, the actual arithmetic intensity of the FC kernel exceeds the maximum theoretical computation throughput of the PUs of the high-performance processor. Therefore, this small deviation does not impact the offloading decision, correctly identifying the FC kernel as compute-bound and ensuring accurate scheduling.

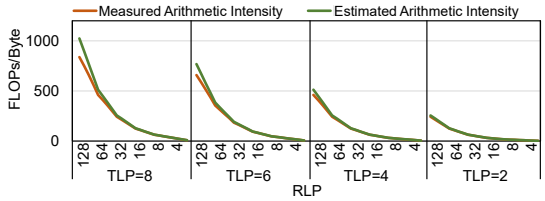

Figure 6. Actual measured arithmetic intensity and the estimated arithmetic intensity for FC kernels in the GPT-3 66B model.

5.2 Runtime Scheduling Implementation

Based on estimated arithmetic intensity, we identify memory-bound or compute-bound FC kernels and dynamically schedule them to the best-fit computing hardware units at runtime. The scheduling process is executed on the host CPU in two steps: (a) initial scheduling and (b) runtime scheduling.

5.2.1 Initial Scheduling. In the initial scheduling step, we decide to offload FC kernels to PUs or FC-PIM units before the LLM serving starts. RLP is set to the batch size, and TLP is set to the system-defined speculation length. We multiply RLP by TLP to estimate the arithmetic intensity and compare it to a memory-boundedness threshold α to make the offloading decision. If the estimated value is larger than α, the FC kernel is estimated as compute-bound and offloaded to PUs; otherwise, it is estimated as memory-bound and executed on FC-PIM units. The threshold α is determined through offline iterative evaluation, where we run the FC kernel on

both PIM and PU units under varying parallelization levels, using the observed execution times to establish the best α to choose.

5.2.2 Runtime Scheduling. In runtime scheduling, we monitor changes in parallelism, predict the current arithmetic intensity, and determine whether or not to reschedule FC kernels to a different computing hardware (i.e., from PUs to FC-PIM units and vice versa). We use a token-level scheduling scheme to track parallelism changes and make real-time decisions based on the estimated arithmetic intensity. The process involves four steps, which we describe next.

First, after each decoding, we gather the output tokens of all requests in the current batch into a single vector. Second, we count the number of <|eos|> tokens in this vector to track changes in *RLP*. If the count is greater than zero, it indicates that some requests have been finished, releasing the corresponding PIM resources allocated to Attn-PIM. *TLP* is typically set initially and does not change frequently at runtime, so we monitor changes in *TLP* with a direct approach: the *TLP* value is stored in a dedicated register, and if the system software running on the host CPU modifies *TLP*, the host CPU notifies (sending instructions) the PAPI system to update the register accordingly. Third, we calculate $RLP \times TLP$ to predict the arithmetic intensity of the next decoding. Fourth, we compare the estimated value with the memory-bound threshold α to decide whether rescheduling FC kernels from PUs/FC-PIM units to FC-PIM/PUs is needed. Figure 5(d) shows an example of our proposed dynamic scheduling technique that enables the execution of LLM decoding on the most suitable hardware units of our proposed architecture based on the real-time demands of the workload, significantly optimizing the performance of LLM inference.

6 PAPI Architecture

6.1 FC-PIM Design

To meet the computation demands of the FC kernel, we need to design a PIM solution with relatively high computation parallelism, while satisfying the necessary power constraints. We modify and use an open-sourced HBM-based PIM simulator [74] that is based on Ramulator 2.0 [43, 57] to evaluate energy consumption and power across different PIM configurations.

We first examine the energy breakdown in a traditional PIM design (e.g., [74]) that integrates one processing core per memory bank, referred to as 1P1B. The energy consumption of PIM execution comes from three parts: *DRAM Access*, *Transfer*, and *Computation*. *DRAM Access* includes the energy consumption required to activate and precharge an HBM DRAM row to read the weight data. *Transfer* includes the energy consumption of transferring activation data from the buffer die, via the TSV, global controller, and bank group controller, to the processing core. *Computation*

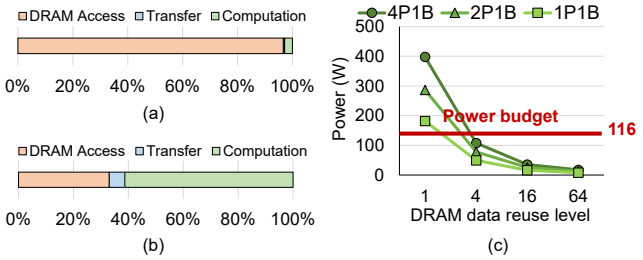

Figure 7. (a) Energy breakdown of PIM for executing the FC kernel with no DRAM data reuse. (b) Energy breakdown of PIM for executing the FC kernel when one DRAM access (i.e., an activated DRAM row) is used 64 times for computation (i.e., data reuse level = 64). (c) Power consumption of PIM architecture with different data reuse levels and different numbers of FPUs per bank.

includes the computation energy in floating point multiplication units (FPUs) of the processing core. As shown in Figure 7(a), most of the energy in PIM execution is consumed by *DRAM Access*, which accounts for 96.7% of the total energy consumption[1] larger than the Q vector (activation data).

Based on the above analysis, accessing data from DRAM *once* and reusing it for *multiple* computations can significantly reduce energy consumption. If data can be accessed from DRAM once and used for multiple computations, the total energy consumption of PIM execution can be reduced significantly. Figure 7(b) shows the energy breakdown of PIM when data is fetched once from DRAM and then reused for 64 FC kernel computations. The energy consumption of *DRAM Access* reduces to 33.1% of the overall energy consumption. This approach gives us a new opportunity to enhance the parallel computation throughput of near-bank PIM. By lowering the energy cost of DRAM access, we gain additional energy budget for the PIM cores.

As described in Section 2, parallelism techniques (batching and speculative decoding) enable data reuse in LLM decoding, which enables parallel PIM execution by allowing the reduction of the *DRAM Access* component. We analyze the power consumption with varying data reuse levels, where a single DRAM access is reused across multiple computations. We explore different PIM configurations, i.e., different numbers of FPUs per DRAM bank. Figure 7(c) shows our results. *xPyB* denotes x FPUs per y banks. The horizontal axis represents the data reuse level, which indicates how many times a single DRAM row is used for FC kernel computations. The vertical axis shows power consumption. We observe that a higher data reuse level leads to a significantly lower power

[1]This energy consumption breakdown is very different from that the HBM-PIM paper [48] reported. The key difference is that the HBM-PIM paper [48] reports *only* the energy consumption breakdown of data movement, while we report the energy consumption breakdown of *both* data movement and computation.

consumption. Specifically, when the data reuse level is ≥ 4, the power consumption of 4P1B becomes significantly lower than that without data reuse (i.e., data reuse = 1) and meets the power budget of HBM.[2] Thus, exploiting data reuse enables the use of more FPUs per DRAM bank while staying within power constraints.

Apart from power constraints, the area constraint of a single HBM die is also a significant barrier against highly parallel PIM designs. Thus, we must ensure that the total HBM-PIM die area including the addtional FPUs stays within the maximum allowable area for a single HBM die. To accommodate additional FPUs within the area-constrained HBM die, we reduce memory capacity, freeing up space for the FPUs. Assuming each PIM-enabled HBM die has m DRAM banks and each DRAM bank employs n FPUs. The total area of memory and FPUs should satisfy the following condition:

$$m(n \times A_{FPU} + A_{bank}) \leq A_{Max} \quad (3)$$

This equation allows us to calculate m to obtain the maximum capacity achievable in a PIM-enabled HBM die using an nP1B PIM configuration.

We use the analytical tool CACTI-3DD [8] to estimate area. The area of one HBM bank A_{bank} is 0.83mm^2 [3] using a 22nm technology node. The area of one HBM die is constrained to 121 mm^2 according to prior work [83]. The area of one FPU A_{FPU} is 0.1025 mm^2 [74]. Thus, the equation for a 4P1B PIM configuration becomes as follows:

$$m(0.1025 \times 4 + 0.83) \leq 121 \quad (4)$$

Therefore, the maximum number of memory banks must be smaller than 97. In our design, we use 96 banks per HBM memory unit, i.e., 3 bank groups (BGs) in the 8-High HBM stack, so as to meet the area constraint of one HBM die with a 4P1B PIM configuration, as shown in Figure 5(b).

6.2 Attn-PIM Design

To address the varying arithmetic intensity, computation demands, and memory footprint of FC and attention kernels, while ensuring high hardware resource utilization, we propose dedicated Attn-PIM units, separate from the FC-PIM units (as described in Section 6.1). The Attn-PIM units are disaggregated from the high-performance processor through an interconnect. This disaggregated design of Attn-PIM allows us to tackle the growing memory footprint demands of KV caches of LLMs, as we explain next.

We find that FC kernels of LLMs have larger computation intensity and result in significantly larger latency than the attention kernels, while attention kernels have larger memory footprint demands. Therefore, given a fixed area budget, FC-PIM requires a configuration with higher computation capability, while the attention kernel does not need as much computation capability. To meet these constraints, we allocate FC-PIM devices with high execution parallelism, i.e., 4 FPUs per DRAM bank (as described in Section 6.1). In contrast, we allocate a larger number of Attn-PIM devices, each of which has lower execution parallelism, using 1 FPU for every two banks, as shown in Figures 5(b) and (c), respectively. Using a single FPU for two banks in Attn-PIM devices ensures that power consumption stays within the HBM power constraints. For attention kernels with a speculation length of 1, a single FPU at 666 MHz with 20.8 MB/s per-bank bandwidth (1P1B) matches the arithmetic intensity of the kernel. However, due to the lack of data reuse in this kernel, the power consumption of 1P1B exceeds the power budget, as shown in Figure 7(c). Consequently, we adopt the 1P2B configuration for each Attn-PIM device to stay within the power consumption limits.

After configuring the FC-PIM and Attn-PIM hardware designs, we determine how many of each of the two types of PIM devices are required for the entire system to efficiently run LLM inference. We configure the total number of PIM devices in the system considering the capacity requirement of LLM inference. The memory capacity requirement for the FC kernel is determined solely by the model size and does not change during runtime. However, the memory capacity required for the attention kernel increases linearly with the sequence length. To support requests with longer sequence lengths, i.e., requests that produce a larger number of output tokens, we disaggregate the Attn-PIM devices from the high-performance processor, which enables accommodating a large number of Attn-PIM devices that can house large memory footprints (in a flexible manner).

Overall, by separately optimizing the parallel computation and memory capacity capabilities of FC-PIM and Attn-PIM devices and having different numbers of devices in the system for the two types of PIM devices we propose, we can satisfy the higher computation and memory bandwidth demands of FC kernels while also satisfying the higher memory capacity and lower computation demands of attention kernels.

6.3 System Integration

Figure 5(a) shows an overview of the interconnection network between the Attn-PIM devices and the high-performance processor and host CPU, where the high-performance processor consists of FC-PIM devices and processing units. FC-PIM devices require high-speed communication with the processing units due to the large volume of weight parameters transferred. Therefore, we select high-speed interconnects like NVLink [66] to connect the FC-PIM devices with the processing units. NVLink provides the required data throughput to ensure that the FC kernels can be executed efficiently without being bottlenecked by data

[2]The power budget of an 8-high, 16GB HBM3 cube is 116 watts [74] following the IDD7 measurement methodology, described in the JEDEC HBM3 specification [37].

[3]The area of bank includes both the memory array and peripheral circuits.

transfer speeds. In contrast, the attention kernel primarily involves small data transfers, such as byte-level Q vector, so a standard interconnect like PCIe (Peripheral Component Interconnect Express) [59] or CXL (Compute Express Link) [15] suffices, depending on the number of devices. PCIe theoretically supports up to 32 devices per bus [61], while CXL can scale to 4,096 devices [15]. These conventional links offer adequate bandwidth for attention kernels and are more cost-effective than high-speed ones.

6.4 Data Partitioning Across PIM Devices

For the attention kernel, we distribute attention heads across Attn-PIM units, with each head assigned to a separate HBM device. We employ the attention mapping scheme from AttAcc [74] on an HBM device, which ensures efficient data movement and parallelism across the PIM architecture. Specifically, the K^T matrix is partitioned column-wise at the pseudo-channel and bank-group levels, and row-wise at the bank and multiplier level. Conversely, the V matrix is partitioned row-wise at the pseudo-channel and bank-group levels, and column-wise at the bank and multiplier level.

For the FC kernel, the large weight matrix is first divided into smaller 2D blocks, each mapped to an HBM device. At the pseudo-channel, bank-group, and bank levels, these weight blocks are partitioned similarly to the K^T matrix in the attention kernel: column-wise at the pseudo-channel and bank-group levels, and row-wise at the bank level.

6.5 Practicality and Architectural Scalability

Complementary PIM Units for Diverse Workloads. We design different FC-PIM and Attn-PIM devices to address distinct computation and memory access patterns in LLMs while maintaining hardware practicality. Both FC-PIM and Attn-PIM devices share the same bank-level computation fabric and memory hierarchy. The key difference lies in the number of processing units (PUs) per bank, which is tailored to the specific computation characteristics of LLM tasks. Attn-PIM, optimized for memory-bound operations, uses fewer PUs per bank to handle memory-intensive tasks efficiently, while FC-PIM is designed for more computation-intensive operations like fully connected (FC) layers, with more PUs per bank to enable higher computation throughput.

The design of Attn-PIM has already been demonstrated to be implementable in industry prototypes and products, such as UPMEM [21, 23–25, 35, 64, 81, 93, 94] and HBM-PIM [46, 48]. Its integration into our system ensures efficient processing of memory-bound tasks, making it a suitable solution for LLM workloads.

FC-PIM, on the other hand, leverages a higher number of PUs per bank to enhance parallel execution and optimize computation throughput for FC layers, while staying within the power limitations of HBM.

By utilizing a shared HBM-PIM computing substrate, both FC-PIM and Attn-PIM benefit from a unified design that avoids modifications to the DRAM core array. Computation logic is embedded within the peripheral circuits, minimizing area overhead while ensuring compatibility with existing HBM technology. We believe this approach simplifies hardware integration and offers scalability, making PIM technology easier to adapt for large-scale deployment in LLM accelerators.

Deployment of Emerging LLM Models. The rapid development of LLMs, particularly Mixture of Experts (MoE) models [18, 50, 88], has introduced new challenges and opportunities for hardware accelerators. MoEs activate only a subset of experts during inference, leveraging sparsity to reduce computation demands. This property is advantageous for hardware accelerators, as it allows for more efficient resource utilization.

FC-PIM is particularly well-suited to exploit the sparsity inherent in MoE architectures. In an MoE model, different experts are activated depending on the input, and the sparsity of these activations presents a significant opportunity to optimize computation. FC-PIM can efficiently execute these sparse operations by storing weight slices from different experts within the same DRAM bank. This allows the system to minimize idle FPUs, which would otherwise remain unused due to the sparsity of MoE models. Moreover, by reducing unnecessary data movement between memory and computation units, FC-PIM helps lower both the energy consumption and the latency associated with MoE inference. These design choices ensure that PAPI can effectively accelerate MoE-based models, making it a viable solution for future LLM architectures.

In summary, the practical implementation of both FC-PIM and Attn-PIM within the PAPI architecture offers a scalable and energy-efficient solution for modern LLM workloads. By leveraging the complementary strengths of these two types of PIM devices and addressing the specific needs of emerging LLM models like MoEs, PAPI is well-positioned to provide high-performance acceleration for a broad range of future LLM applications.

7 Evaluation

7.1 Evaluation Methodology

Comparison Points and Simulation Methodology. We compare PAPI with three state-of-the-art systems: (a) *A100+AttAcc:* a heterogeneous computing platform with 6 NVIDIA A100 GPUs [12] and AttAcc PIM-based units (one FPU unit per DRAM bank, i.e., 1P1B configuration), which is the state-of-the-art design proposed by prior work [74]. All FC kernel computations are executed on GPUs, and attention kernel computations are handled by AttAcc PIM-based units; (b) *A100+HBM-PIM:* an integrated computing platform with 6 NVIDIA A100 GPUs and HBM-PIM devices. HBM-PIM [48] is a commercial PIM device produced by Samsung, featuring one FPU unit per 2 DRAM banks (i.e., 1P2B configuration);

(c) *AttAcc-only:* a PIM-only computing platform with AttAcc PIM-based units [74], in which all computations of FC and attention kernels are executed on PIM units. In the PAPI design, the capacity of the FC-PIM devices is 12 GB, while all other HBM devices, including Attn-PIM devices in PAPI, have a capacity of 16 GB. Therefore, one GPU Memory in PAPI is 60 GB rather than 80 GB in the A100 GPU, necessitating six GPUs to accommodate the model parameters of GPT-3 175B (requiring 350 GB memory). For a fair comparison, each of the computing systems has 90 HBM devices, 30 for storing the weight parameters of FC kernels and 60 for attention kernels. Each GPU contains 5 HBM devices connected via NVLink [66], corresponding to the 80GB GPU memory of the A100 GPU. All HBMs used in the experiments are HBM3 [37] with 5.2Gbps per pin and running at 333MHz. We developed a simulator based on Ramulator2 [57] (new version of Ramulator [43]) and AttAcc [74] to evaluate the performance and energy efficiency of the PAPI computing platform, including both GPU and PIM-based components.

Workloads. We evaluate three transformer-based LLMs, LLaMA-65B [92], GPT-3 66B [6], and GPT-3 175B [6], using the FP16 data type. We use creative-writing and general-qa tasks in the Dolly dataset [14]. The Dolly dataset is an open-source dataset of instruction-following records generated by thousands of Databricks employees in several behavioral categories outlined in InstructGPT [72]. We use static batching with varying initial request-level parallelism (batch size) across experiments. By evaluating our proposed design on real-world datasets, we can test the performance and energy consumption with various input and output sequence lengths while adapting to dynamic parallelization levels observed at runtime.

7.2 End-to-End Performance and Energy Efficiency

Performance Speedup. Figure 8(a) shows the end-to-end performance of all four evaluated designs using various parallelization levels in the decoding step of each model with batch sizes of 4, 16, or 64 and speculation lengths of 1, 2, or 4. The results are normalized to the A100+AttAcc baseline.

We make three observations. First, PAPI achieves speedups of 1.8×, 1.9×, and 11.1× over A100+AttAcc, A100+HBM-PIM, and AttAcc-only designs, respectively. This is because PAPI schedules tasks between GPU and PIM *dynamically*, offloading each task to the corresponding best-fit computing hardware at a given point in time, and the proposed hybrid PIM architecture can provide varying levels of execution parallelism, catering to the different needs of the FC and attention kernels. Second, the AttAcc-only scheme performs worse than PAPI and also worse than A100+AttAcc at most parallelization settings. This is due to two reasons. (1) AttAcc-only has limited computation throughput as it employs solely PIM units. Later (Section 7.4), we compare the performance of PIM solutions with different parallel computation capabilities. (2) FC kernels with the parallelism settings used in our experiments are more computation-intensive, making them unable to benefit from a PIM-only solution (which is a better fit for memory-intensive kernels). Third, A100+AttAcc performs similarly to A100+HBM-PIM because the only difference between them is the execution of the attention kernel on either AttAcc or HBM-PIM. However, the attention kernel's execution time on PIM is relatively small compared to the overall runtime, resulting in a small performance difference.

Figure 9(a) illustrates the end-to-end latency of three designs on the Dolly general-qa dataset. PAPI achieves speedups of 1.7×, 1.7×, and 8.1× over A100+AttAcc, A100+HBM-PIM and AttAcc-only, respectively, which is lower than the speedup for the Dolly creative-writing dataset. This is due to two reasons: (i) The creative-writing dataset typically has longer output lengths, which makes the decoding phase a larger bottleneck for end-to-end performance, thereby making PAPI acceleration more beneficial. (ii) Longer output lengths of the creative-writing dataset lead to more significant dynamic changes in parallelization levels,

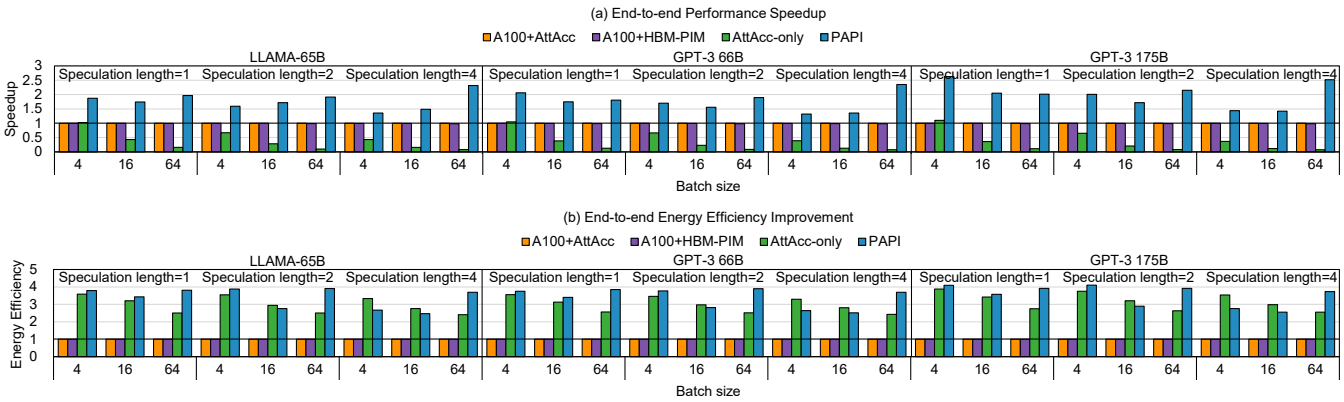

Figure 8. End-to-end speedup (top) and energy efficiency (bottom) comparisons of four evaluated designs on the Dolly creative-writing dataset. Values are normalized to A100+AttAcc.

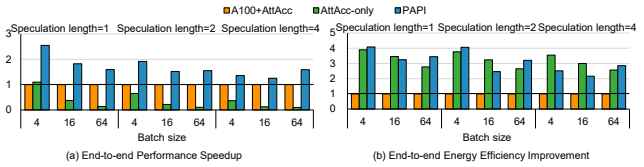

Figure 9. End-to-end speedup (a) and energy efficiency (b) comparisons of three evaluated designs on the Dolly general-qa dataset for GPT-3 175B.

thereby allowing PAPI to further improve performance over prior schemes.

We conclude that PAPI provides significant performance benefits in LLM inference over state-of-the-art PIM-based designs across various real-world configuration settings (speculation length, batch size) and using different real datasets.

Energy Efficiency. Figures 8(b) and 9 (b) present the end-to-end energy efficiency, normalized to A100+AttAcc system, for the creative-writing and general-qa datasets. PAPI improves average energy efficiency by 3.4× and 3.1× for these datasets, respectively, over A100+AttAcc. This is because A100+AttAcc executes the FC kernels on energy-hungry A100 GPUs, while PAPI offloads parts of these kernels to FC-PIM devices, thereby consuming less energy by mitigating data movement and exploiting low-power processing cores in memory. Compared to AttAcc-only, PAPI provides 1.15× and 1.01× energy efficiency improvement in creative-writing and general-qa datasets, respectively, which is lower than PAPI benefits over A100+AttAcc. This is because PAPI dynamically schedules the FC kernels on the energy-hungry GPU cores and the energy-efficient PIM cores. While GPU execution consumes more energy than AttAcc-only, PAPI lowers energy consumption on PIM through DRAM data access reuse, resulting in modest savings over AttAcc-only.

We conclude that PAPI improves energy efficiency over state-of-the-art PIM systems across different real configuration settings and datasets.

7.3 Sensitivity to Parallelization Levels

We analyze the performance of three evaluated designs across different RLP and TLP values, using the LLaMA-65B model on the creative-writing dataset.

RLP. Figure 10(a) presents the performance of three designs when we vary the batch size from 4 to 128 to explore the effect of RLP, using a fixed speculation length of 1. When RLP is relatively low, e.g., with a batch size of 4, AttAcc-only provides higher performance than A100+AttAcc. As RLP increases, the execution time of AttAcc-only increases significantly because the PIM devices cannot effectively cater to the large computation needs of the FC kernels, which leads to AttAcc-only providing much worse performance than A100+AttAcc. PAPI achieves the best performance for all RLP settings over state-of-the-art PIM-based systems.

TLP. Figure 10(b) shows the performance of three designs when we vary the speculation length from 1 to 8 to analyze various TLP levels, using a fixed batch size of 4. Compared to A100+AttAcc and AttAcc-only, PAPI achieves 1.5× and 3.0× speedup on average. The speedup of PAPI over A100+AttAcc decreases as TLP increases, because PAPI offloads more FC kernels to the GPUs as TLP increases. If TLP becomes large enough, we expect that PAPI would assign all the FC kernels to the GPU and thus the performance of PAPI to converge to that of A100+AttAcc.

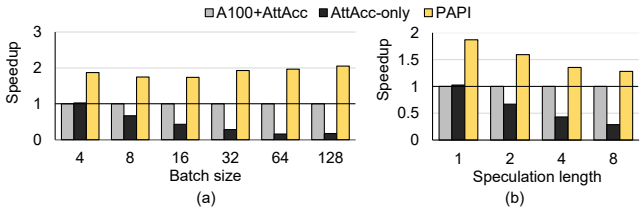

Figure 10. End-to-end speedup with (a) different batch sizes (speculation length=1), and (b) different speculation lengths (batch size=4); for LLaMA-65B.

7.4 Performance Analysis of PAPI

To analyze the benefits of our proposed hybrid PIM design in PAPI, we compare the performance of two PIM-only architectures: (i) AttAcc-only and (ii) our proposed PIM architecture with only Attn-PIM and FC-PIM devices (but without the A100), using the same number of PIM devices and interconnect settings for fairness. We only evaluate the decoding phase (since the prefilling phase is compute-bound and is to be executed on the GPU platform). Figure 11 shows the speedup of our PIM-only PAPI design compared to AttAcc-only using the Dolly creative-writing dataset. Our PIM design achieves 2.3× speedup improvement against AttAcc-only on average. We observe that our PIM design has a higher speedup at higher parallelization levels: e.g., when the batch size is 4 and the speculation length is 1, the speedup is 1.6×, while when the batch size is 64 and the speculation length is 4 (higher parallelism) the speedup of PIM-only PAPI increases to 2.7×. This is because, as parallelism increases, FC kernels become more computation-intensive, requiring more computation power. PAPI with more processing units (PUs) can provide the required computation capability much more so than AttAcc-only. Additionally, FC kernels are responsible for most of the execution time, so improving their performance has the largest impact on overall speedup.

Figure 12 presents the execution time breakdown per token for the AttAcc-only system and for the PIM-only PAPI system with Attn-PIM and FC-PIM devices. We make four key observations. First, FC kernels dominate the total execution time. Therefore, it is valuable to enable higher execution parallelism in PIM hardware (as PAPI does with FC-PIM)

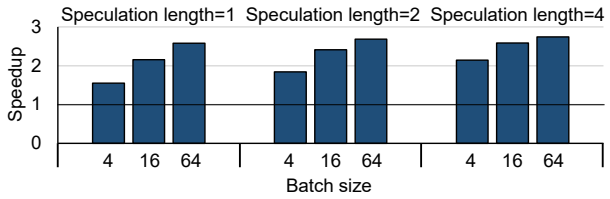

Figure 11. Performance speedup of PIM-only PAPI over AttAcc-only in the decoding phase for the Dolly creative-writing dataset.

to effectively cater to the high computation demands of the FC kernels. Second, the PIM-only PAPI design provides 2.9× speedup when processing FC kernels. Third, attention kernels run 1.7× slower on Attn-PIM (1P2B) than AttAcc-only (1P1B) due to our design choice that reduces FPU area overheads. Fourth, communication takes up 28.2% of the total execution time in the decoding stage; thus, more advanced network technologies could be developed and integrated into the PAPI architecture to further improve performance.

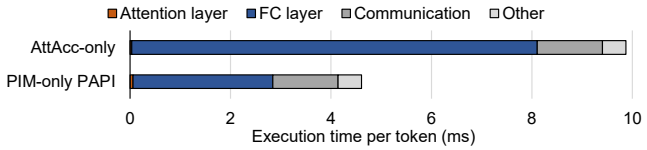

Figure 12. Execution time breakdown per token in the decoding phase of LLaMA-65B model inference (batch size=4, speculation length=4) for AttAcc-only versus PIM-only PAPI.

8 Related Work

To our knowledge, PAPI provides the first architecture and a runtime framework to tackle dynamically varying parallelization levels and hence dynamically varying computation and memory demands of real-world LLM workloads. We comprehensively compare PAPI to two state-of-the-art PIM designs, AttAcc [74] and HBM-PIM [48], demonstrating PAPI's significant performance and energy benefits over them (Section 7.2).

PIM-enabled LLM accelerators. The PIM computing paradigm [62] addresses the data movement bottleneck between memory and processors by placing computation near or inside memory circuitry. For transformer-based LLMs, PIM (e.g., [29, 36, 40, 45, 46, 48, 49, 53, 75, 99, 103]) provides a promising opportunity to accelerate the memory-bound kernels in the decoding phase. DRAM-based PIM [29, 63], with its large memory capacity and bandwidth, is particularly well-suited for LLMs. For example, the SK Hynix AiM PIM architecture [45] offloads both FC and attention kernels to GDDR6-PIM accelerators, outperforming A100 GPUs in single-batch scenarios. However, the architecture performs poorly when FC kernels are compute-bound, e.g., with larger batch sizes.

Prior works propose heterogenous PIM-enabled computing systems for LLM inference. AttAcc [74] proposes an HBM-based PIM architecture for attention kernels while running the FC kernels on GPUs to accelerate LLM inference with large batch sizes. Section 7.2 shows that PAPI outperforms this scheme by designing a more effective PIM-based architecture carefully tailored to the dynamically varying computation and memory needs of FC and attention kernels. IANUS [84] offloads *all* FC kernels to PIM to efficiently handle non-batched requests. This would provide low performance in scenarios involving batched requests, which are common in real-world LLM inference. SpecPIM [52] proposes a PIM-enabled system with NPUs and PIM cores, leveraging speculative decoding. It introduces a decoding parallelism-aware scheduling method based on a genetic algorithm and Monte Carlo Tree Search (MCTS). This offline scheduling process involves 50 rounds of the genetic algorithm and 10,000 leaf node searches for MCTS. While this scheduling method provides performance benefits in cases with a fixed batch size and speculation length, its computational complexity makes it impractical for dynamic execution. In dynamic real-world LLM inference scenarios, especially when decoding parallelism levels vary over time, SpecPIM would need to repeatedly run MTCS scheduling, incurring high-performance costs.

Other LLM accelerators. Prior works explore hardware LLM accelerators to improve LLM inference performance. DFX [34] introduces a multi-FPGA accelerator with high-bandwidth memory (HBM) for end-to-end inference acceleration, and provides an efficient dataflow when the decoding stage is memory-bound. However, even when using HBM, such designs still suffer from the memory bottleneck, especially when attention kernels exhibit very low arithmetic intensity [98]. AMX-GPU [42] proposes an adaptive LLM model scheduling strategy for CPU-GPU cooperative computing. While this design can adapt to different batch sizes and token lengths, it does not account for runtime changes in parallelism, such as varying concurrency of requests or dynamic changes in computation versus memory bottlenecks.

Recent research utilizes various approximation algorithms, like pruning and quantization, to reduce the amount of data movement (e.g., [16, 17, 26–28, 41, 56, 77, 96, 100]). For example, SpAtten [97] introduces token pruning to remove unimportant tokens during inference. These approximation approaches are suitable for LLM scenarios that can tolerate approximate results. PAPI does not sacrifice quality in LLM serving, while providing significant performance and energy benefits over state-of-the-art systems.

9 Conclusion

Real-world LLM services with state-of-the-art parallelism optimization techniques, such as batching and speculation decoding, lead to dynamically-changing parallelization levels. As a result, fully-connected and attention kernels in LLM inference exhibit varying computation and memory demands. To seamlessly adapt to such dynamic demands, we propose PAPI, a computing system that supports three types of computing units with different computation and memory bandwidth capabilities, and a lightweight scheduling framework that offloads fully-connected and attention kernels to the most suitable computing units by monitoring the dynamic parallelization levels in LLM inference at low cost. Our evaluation shows that PAPI provides 1.8× and 11.1× performance improvement over state-of-the-art LLM inference systems. We hope that our work enables further research on leveraging heterogeneous PIM-enabled systems to cater to dynamic real-world execution scenarios in emerging machine learning models such as LLMs.

Acknowledgments

We sincerely thank the anonymous reviewers of ASPLOS 2025 for feedback. We thank the SAFARI group members for feedback and the stimulating intellectual environment they provide. This work was supported by the National Natural Science Foundation of China (Grant No. 62090024, 62222411), the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDB0660100, and the National Key R&D Program of China, Grant No. 2023YFB4404400. Ying Wang and Huawei Li are the corresponding authors (wangying2009@ict.ac.cn, lihuawei@ict.ac.cn). We acknowledge the generous gifts from our industrial partners, including Google, Huawei, Intel, and Microsoft. This work is supported in part by the ETH Future Computing Laboratory (EFCL), Huawei ZRC Storage Team, Semiconductor Research Corporation, AI Chip Center for Emerging Smart Systems (ACCESS), sponsored by InnoHK funding, Hong Kong SAR, and European Union's Horizon programme for research and innovation [101047160 - BioPIM].

References

[1] Shaizeen Aga, Supreet Jeloka, Arun Subramaniyan, Satish Narayanasamy, David Blaauw, and Reetuparna Das. 2017. Compute Caches. In *HPCA*.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *OSDI*.

[3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).

[4] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing. In *ISCA*.

[5] Yushi Bai, Jiajie Zhang, Xin Lv, Linzhi Zheng, Siqi Zhu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2025. LongWriter: Unleashing 10,000+ Word Generation from Long Context LLMs. In *ICLR*.

[6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. *NeurIPS*.

[7] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318* (2023).

[8] Ke Chen, Sheng Li, Naveen Muralimanohar, Jung Ho Ahn, Jay B Brockman, and Norman P Jouppi. 2012. CACTI-3DD: Architecture-level Modeling for 3D Die-stacked DRAM Main Memory. In *DATE*.

[9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[10] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. DianNao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning. In *ASPLOS*.

[11] Jaewan Choi, Jaehyun Park, Kwanhee Kyung, Nam Sung Kim, and Jung Ho Ahn. 2023. Unleashing the Potential of PIM: Accelerating Large Batched Inference of Transformer-Based Generative Models. In *IEEE CAL*.

[12] Jack Choquette and Wish Gandhi. 2020. NVIDIA A100 GPU: Performance & Innovation for GPU Computing. In *Hot Chips*.

[13] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. PaLM: Scaling Language Modeling with Pathways. In *JMLR*.

[14] Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM. In *Company Blog of Databricks*.

[15] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. In *ACM Computing Surveys*.

[16] Jyotikrishna Dass, Shang Wu, Huihong Shi, Chaojian Li, Zhifan Ye, Zhongfeng Wang, and Yingyan Lin. 2023. ViTALiTy: Unifying Low-rank and Sparse Approximation for Vision Transformer Acceleration with a Linear Taylor Attention. In *HPCA*.

[17] Peiyan Dong, Mengshu Sun, Alec Lu, Yanyue Xie, Kenneth Liu, Zhenglun Kong, Xin Meng, Zhengang Li, Xue Lin, Zhenman Fang, et al. 2023. HeatViT: Hardware-Efficient Adaptive Token Pruning for Vision Transformers. In *HPCA*.

[18] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. In *JMLR*.

[19] João Dinis Ferreira, Gabriel Falcao, Juan Gómez-Luna, Mohammed Alser, Lois Orosa, Mohammad Sadrosadati, Jeremie S Kim, Geraldo F Oliveira, Taha Shahroodi, Anant Nori, et al. 2022. pLUTo: Enabling Massively Parallel Computation in DRAM via Lookup Tables. In *MICRO*.

[20] Bin Gao, Zhehui Wang, Zhuomin He, Tao Luo, Weng-Fai Wong, and Zhi Zhou. 2024. IMI: In-memory Multi-job Inference Acceleration for Large Language Models. In *ICPP*.

[21] Christina Giannoula, Ivan Fernandez, Juan Gómez Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. 2022. SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures. In *Proceedings of the ACM on Measurement and Analysis of Computing Systems*.

[22] Christina Giannoula, Peiming Yang, Ivan Fernandez, Jiacheng Yang, Sankeerth Durvasula, Yu Xin Li, Mohammad Sadrosadati, Juan Gomez Luna, Onur Mutlu, and Gennady Pekhimenko. 2024. PyGim: An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* (2024).

[23] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F Oliveira, and Onur Mutlu. 2021. Benchmarking Memory-Centric Computing Systems: Analysis of Real Processing-in-Memory Hardware. In *CUT*.

[24] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F Oliveira, and Onur Mutlu. 2022. Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System. In *IEEE Access*.

[25] Juan Gómez-Luna, Yuxin Guo, Sylvan Brocard, Julien Legriel, Remy Cimadomo, Geraldo F Oliveira, Gagandeep Singh, and Onur Mutlu. 2023. Evaluating Machine LearningWorkloads on Memory-Centric Computing Systems. In *ISPASS*.

[26] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2023. OliVe: Accelerating Large Language Models via Hardware-friendly Outlier-Victim Pair Quantization. In *ISCA*.

[27] Tae Jun Ham, Sung Jun Jung, Seonghak Kim, Young H Oh, Yeonhong Park, Yoonho Song, Jung-Hun Park, Sanghee Lee, Kyoung Park, Jae W Lee, et al. 2020. A^3: Accelerating Attention Mechanisms in Neural Networks with Approximation. In *HPCA*.

[28] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, Soosung Kim, Hyunji Choi, Sung Jun Jung, and Jae W Lee. 2021. ELSA: Hardware-Software Co-design for Efficient, Lightweight Self-Attention Mechanism in Neural Networks. In *ISCA*.

[29] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and TN Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *MICRO*.

[30] Yintao He, Ying Wang, Cheng Liu, Huawei Li, and Xiaowei Li. 2021. TARe: Task-Adaptive in-situ ReRAM Computing for Graph Learning. In *DAC*.

[31] Yintao He, Ying Wang, Yongchen Wang, Huawei Li, and Xiaowei Li. 2019. An Agile Precision-Tunable CNN Accelerator based on ReRAM. In *ICCAD*.

[32] Yintao He, Ying Wang, Xiandong Zhao, Huawei Li, and Xiaowei Li. 2020. Towards State-Aware Computation in ReRAM Neural Networks. In *DAC*.

[33] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In *ASPLOS*.

[34] Seongmin Hong, Seungjae Moon, Junsoo Kim, Sungjae Lee, Minsub Kim, Dongsoo Lee, and Joo-Young Kim. 2022. DFX: A Low-latency Multi-FPGA Appliance for Accelerating Transformer-based Text Generation. In *MICRO*.

[35] Bongjoon Hyun, Taehun Kim, Dongjae Lee, and Minsoo Rhu. 2024. Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology. In *HPCA*.

[36] Hongsun Jang, Jaeyong Song, Jaewon Jung, Jaeyoung Park, Youngsok Kim, and Jinho Lee. 2024. Smart-Infinity: Fast Large Language Model Training using Near-Storage Processing on a Real System. In *HPCA*.

[37] JEDEC. 2022. High Bandwidth Memory DRAM (HBM3). (2022).

[38] Taeyang Jeong and Eui-Young Chung. 2024. PipePIM: Maximizing Computing Unit Utilization in ML-Oriented Digital PIM by Pipelining and Dual Buffering. In *IEEE TCAD*.

[39] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *ISCA*.

[40] Hongju Kal, Chanyoung Yoo, and Won Woo Ro. 2023. AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory. In *MICRO*.

[41] Sheng-Chun Kao, Suvinay Subramanian, Gaurav Agrawal, Amir Yazdanbakhsh, and Tushar Krishna. 2023. FLAT: An Optimized Dataflow for Mitigating Attention Bottlenecks. In *ASPLOS*.

[42] Hyungyo Kim, Gaohan Ye, Nachuan Wang, Amir Yazdanbakhsh, and Nam Sung Kim. 2024. Exploiting Intel® Advanced Matrix Extensions (AMX) for Large Language Model Inference. In *IEEE CAL*.

[43] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A Fast and Extensible DRAM Simulator. In *IEEE CAL*.

[44] Hyucksung Kwon, Kyungmo Koo, Janghyeon Kim, Woongkyu Lee, Minjae Lee, Hyungdeok Lee, Yousub Jung, Jaehan Park, Yosub Song, Byeongsu Yang, et al. 2024. LoL-PIM: Long-Context LLM Decoding with Scalable DRAM-PIM System. In *arXiv preprint arXiv:2412.20166*.

[45] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, et al. 2022. System Architecture and Software Stack for GDDR6-AiM. In *HCS*.

[46] Young-Cheon Kwon, Suk Han Lee, Jaehoon Lee, Sang-Hyuk Kwon, Je Min Ryu, Jong-Pil Son, O Seongil, Hak-Soo Yu, Haesuk Lee, Soo Young Kim, et al. 2021. 25.4 A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications. In *ISSCC*.

[47] Hyungdeok Lee, Guhyun Kim, Dayeon Yun, Ilkon Kim, Yongkee Kwon, and Euicheol Lim. 2024. Cost-Effective LLM Accelerator Using Processing in Memory Technology. In *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*.

[48] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, et al. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product. In *ISCA*.

[49] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, et al. 2022. A 1ynm 1.25V 8Gb, 16Gb/s/pin GDDR6-Based Accelerator-in-Memory Supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications. In *ISSCC*.

[50] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *ICLR*.

[51] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *ICML*.

[52] Cong Li, Zhe Zhou, Size Zheng, Jiaxi Zhang, Yun Liang, and Guangyu Sun. 2024. SpecPIM: Accelerating Speculative Inference on PIM-Enabled System via Architecture-Dataflow Co-Exploration. In *ASPLOS*.

[53] Huize Li, Zhaoying Li, Zhenyu Bai, and Tulika Mitra. 2024. ASADI: Accelerating Sparse Attention Using Diagonal-based In-Situ Computing. In *HPCA*.

[54] Shuangchen Li, Dimin Niu, Krishna T Malladi, Hongzhong Zheng, Bob Brennan, and Yuan Xie. 2017. DRISA: A DRAM-Based Reconfigurable In-Situ Accelerator. In *MICRO*.

[55] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *OSDI*. 663–679.

[56] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. 2021. Sanger: A Co-Design Framework for Enabling Sparse Attention using Reconfigurable Architecture. In *MICRO*.

[57] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkçı, and Onur Mutlu. 2023. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. In *IEEE CAL*.

[58] Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nadav Timor, Moshe Wasserblat, and Roy Schwartz. 2024. Accelerating Speculative Decoding using Dynamic Speculation Length. *arXiv preprint arXiv:2405.04304* (2024).

[59] David Mayhew and Venkata Krishnan. 2003. PCI Express and Advanced Switching: Evolutionary path to building next generation interconnects. In *11th Symposium on High Performance Interconnects*.

[60] Microsoft. 2023. Copilot. https://github.com/features/copilot.

[61] David J Miller, Philip M Watts, and Andrew W Moore. 2009. Motivating Future Interconnects: a Differential Measurement Analysis of PCI Latency. In *ANCS*.

[62] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. 2022. A Modern Primer on Processing in Memory. In *Emerging computing: from devices to systems: looking beyond Moore and Von Neumann*.

[63] Onur Mutlu, Ataberk Olgun, Geraldo F Oliveira, and Ismail Emir Yuksel. 2024. Memory-Centric Computing: Recent Advances in Processing-in-DRAM. In *IEDM*.

[64] Joel Nider, Craig Mustard, Andrada Zoltan, John Ramsden, Larry Liu, Jacob Grossbard, Mohammad Dashti, Romaric Jodin, Alexandre Ghiti, Jordi Chauzi, et al. 2021. A Case Study of Processing-in-Memory in off-the-Shelf Systems. In *USENIX ATC 21*.

[65] NVIDIA. 2016. Triton Inference Server. https://developer.nvidia.com/triton-inference-server. (2016).

[66] NVIDIA. 2017. NVLink. http://www.nvidia.com/object/nvlink.html.

[67] NVIDIA. 2020. NVIDIA A100 Tensor Core GPU Architecture. White Paper. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf.

[68] NVIDIA. 2023. DGX A100. https://resources.nvidia.com/en-us-dgx-systems/dgx-ai.

[69] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. 2024. ExeGPT: Constraint-Aware Resource Scheduling for LLM Inference. In *ASPLOS*.

[70] OpenAI. 2024. Sora: Creating Video from Text. https://openai.com/sora.

[71] Cristobal Ortega, Yann Falevoz, and Renaud Ayrignac. 2024. PIM-AI: A Novel Architecture for High-Efficiency LLM Inference. *arXiv preprint arXiv:2411.17309* (2024).

[72] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *NeurIPS*.

[73] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. 2024. InstInfer: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference. *arXiv preprint arXiv:2409.04992* (2024).

[74] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference. In *ASPLOS*.

[75] Sang-Soo Park, KyungSoo Kim, Jinin So, Jin Jung, Jonggeon Lee, Kyoungwan Woo, Nayeon Kim, Younghyun Lee, Hyungyo Kim, Yongsuk Kwon, et al. 2024. An LPDDR-based CXL-PNM Platform for TCO-efficient Inference of Transformer-based Large Language Models. In *HPCA*.

[76] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ISCA*.

[77] Zheng Qu, Liu Liu, Fengbin Tu, Zhaodong Chen, Yufei Ding, and Yuan Xie. 2022. DOTA: Detect and Omit Weak Attentions for Scalable Transformer Acceleration. In *ASPLOS*.

[78] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *ICML*.

[79] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language Models are Unsupervised Multitask Learners. In *OpenAI blog*.

[80] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. 2021. Zero-Shot Text-to-Image Generation. In *ICML*.

[81] Steve Rhyner, Haocong Luo, Juan Gómez-Luna, Mohammad Sadrosadati, Jiawei Jiang, Ataberk Olgun, Harshita Gupta, Ce Zhang, and Onur Mutlu. 2024. PIM-Opt: Demystifying Distributed Optimization Algorithms on a Real-World Processing-In-Memory System. In *PACT*.

[82] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis with Latent Diffusion Models. In *CVPR*.

[83] Yesin Ryu, Sung-Gi Ahn, Jae Hoon Lee, Jaewon Park, Yong Ki Kim, Hyochang Kim, Yeong Geol Song, Han-Won Cho, Sunghye Cho, Seung Ho Song, et al. 2023. A 16 GB 1024 GB/s HBM3 DRAM With Source-Synchronized Bus Design and On-Die Error Control Scheme for Enhanced RAS Features. In *IEEE JSSC*.

[84] Minseok Seo, Xuan Truong Nguyen, Seok Joong Hwang, Yongkee Kwon, Guhyun Kim, Chanwook Park, Ilkon Kim, Jaehan Park, Jeongbin Kim, Woojae Shin, et al. 2024. IANUS: Integrated Accelerator based on NPU-PIM Unified Memory System. In *ASPLOS*.

[85] Tensorflow serving. 2023. Tensorflow. https://github.com/tensorflow/serving.

[86] Vivek Seshadri, Kevin Hsieh, Amirali Boroum, Donghyuk Lee, Michael A Kozuch, Onur Mutlu, Phillip B Gibbons, and Todd C Mowry. 2015. Fast Bulk Bitwise AND and OR in DRAM. In *IEEE CAL*.

[87] Vivek Seshadri, Donghyuk Lee, Thomas Mullins, Hasan Hassan, Amirali Boroumand, Jeremie Kim, Michael A Kozuch, Onur Mutlu, Phillip B Gibbons, and Todd C Mowry. 2017. Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology. In *MICRO*.

[88] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2016. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *ICLR*.

[89] Haihao Shen, Hanwen Chang, Bo Dong, Yu Luo, and Hengyu Meng. 2023. Efficient LLM inference on CPUs. In *NeurIPS Workshop*.

[90] Benjamin Spector and Chris Re. 2023. Accelerating LLM Inference with Staged Speculative Decoding. In *ICML Workshop*.

[91] Qidong Su, Christina Giannoula, and Gennady Pekhimenko. 2023. The Synergy of Speculative Decoding and Batching in Serving Large Language Models. *arXiv preprint arXiv:2310.18813* (2023).

[92] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).

[93] UPMEM. 2018. Introduction to UPMEM PIM. Processing-In-Memory (PIM) on DRAM Accelerator (White Paper).

[94] UPMEM. 2020. UPMEM Website. https://www.upmem.com.

[95] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *NeurIPS*.

[96] Haoran Wang, Haobo Xu, Ying Wang, and Yinhe Han. 2023. CTA: Hardware-Software Co-design for Compressed Token Attention Mechanism. In *HPCA*.

[97] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. In *HPCA*.

[98] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. 2023. Flash-LLM: Enabling Cost-Effective and Highly-Efficient Large Generative Model Inference with Unstructured Sparsity. In *VLDB*.

[99] Amir Yazdanbakhsh, Ashkan Moradifirouzabadi, Zheng Li, and Mingu Kang. 2022. Sparse Attention Acceleration with Synergistic In-Memory Pruning and On-Chip Recomputation. In *MICRO*.

[100] Haoran You, Zhanyi Sun, Huihong Shi, Zhongzhi Yu, Yang Zhao, Yongan Zhang, Chaojian Li, Baopu Li, and Yingyan Lin. 2023. ViT-CoD: Vision Transformer Acceleration via Dedicated Algorithm and Accelerator Co-Design. In *HPCA*.

[101] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. ORCA: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI*.

[102] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024. Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. In *ACL*.

[103] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A Memory-based Acceleration via Software-Hardware Co-Design for Transformer. In *HPCA*.

[104] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. 2024. A Survey on Efficient Inference for Large Language Models. *arXiv preprint arXiv:2404.14294* (2024).